Group 2613 – Assignment 2: CNNs

Giacomo Bregolin, Leonardo Fardin, Lara Fogarolo, and Francesco Peron
(Dated: May 21, 2026)

Signal detection techniques have a lot of practical application in many fields of Physics: first of all, GW detection. Neural Networks, and specifically CNNs, are becoming an increasingly popular tool for this purpose. In this paper, we intend to study the generalization capacity of a specific CNN over a specific dataset, which basically simulates a known-shape signal. By doing this, we can understand how to train a CNN for detecting signals in an effective way. In order to do that, we show that the dataset's Signal-to-Noise Ratio (SNR) provides an effective measure for the CNN's test accuracy and learning potential over it. Using this measure, we show that the best generalization capacity over noisy datasets is obtained by training the CNN over a noisy dataset. We have studied how fast the CNN converges to a fixed test accuracy depending on the dataset's noisiness and we finally studied how the regularization has an impact on the CNN's generalization capacity.

INTRODUCTION

Memorization vs pattern learning in CNNs – Neural Networks are usually over-parametrized, this could bring the NN to memorize the data instead of learning the underlying pattern. Arpit et al. [1] showed that regularization and well-structured datasets could help the NN to learn better the pattern and thus generalize better to real data. The intent of this article is to study the pattern learning for the specific dataset and CNN given, respectfully in Methods: Data Generation and Appendix B.

CNN's sensitivity – The CNN used works as a signal detector, specifically as a Matched Filter[2] (a Self-Learning Matched Filter since it learns autonomously the pattern to find). Adopting the tools provided by the *Signal Detection Theory*, we can compute an upper limit to the *Signal-To-Noise Ratio* (SNR) seen by the CNN:

$$SNR_{MF} = \frac{\pi^3}{2(\pi+4)}\frac{\delta}{\sigma^2} \tag{1}$$

Where $\delta = \frac{A^2}{T}$, A the amplitude of the signal, T its duration and σ the noise's stdev. The way in which the equations were derived is reported in Appendix A.

In the *Methods* section we will demonstrate that the computed SNR is a good measure of both the classifiability of the dataset and the CNN's learning potential over it.

Objectives of the work The purpose of this article is:

1. Define a measure for the classifiability of a dataset and the CNN's learning potential over it.
2. Generalize the results to datasets with higher- dimensional feature spaces.
3. Test the CNN's performance in terms of convergence time and accuracy over datasets with varying SNR.
4. Test the CNN's generalization capacity with varying SNR.
5. Study how regularization affects the CNN's learning process.

METHODS

Data generation – Each dataset consists of 2-channel time series of $N = 10000$ data points, generated from stochastic random-walk processes with the addition of localized sinusoidal patterns. Each sample has length $L = 60$ and belongs to one of 3 classes:

- 0 - pure random walk
- 1 - positive sinusoidal pattern injected
- 2 - negative sinusoidal pattern inject

The injected pattern added at a random temporal $j_0^{(k)}$ in channel k is:

$$s_k(i) = A_k sin\left(\frac{\pi i}{d}\right) \tag{2}$$

where A_k is the channel-specific amplitude and $d = 11$ is the duration of the signal.

The background signal in each channel is generated as a biased random walk:

$$x_t = x_{t-1} + \epsilon_t \quad with \quad \epsilon_t \sim \mathcal{N}(b, \sigma^2) \tag{3}$$

where $b = 5$ and $\sigma^2 = 20$.

Validating the SNR as a measure of learning potential – We define eleven SNR values and we compute five pairs (A, σ) for each, yielding that same SNR. Remember that T is fixed because of the CNN's structure. For each pair, five models are trained and their highest validation accuracy value is stored. We then compute for each pair the mean and the error of the maximum validation accuracy.

We test the null hypothesis that pairs (A, σ) with equal SNR yield the same accuracy distribution. Failure to reject this hypothesis would confirm that the computed SNR is a reliable metric for the dataset's learning potential. We use the chi-square variable defined as:

$$\chi^2 = \sum_g \sum_i \frac{(y_i - \mu_g)^2}{\sigma_i^2 + \sigma_g^2} \tag{4}$$

where g stands for the group of points with the same SNR, μ_g is the mean accuracy of the group, and y_i is the accuracy of the i-th element of group g.

Generalization of the validity of SNR in higher dimension feature spaces – The theory tells us that, in the optimal case, the total SNR is given by the sum of the SNR on each feature (linearity of integrals and scalar product). We repeat the procedure used in 1-D for the bi-dimensional case as well. In this case 6 SNR values were evaluated. The test in this case is then:

$$\chi^2 = \sum_g \sum_i \frac{(y_i - \mu_g)^2}{\sigma_i^2 + \sigma_g^2} \tag{5}$$

where g stands for the group of points with the same total SNR, μ_g is the mean accuracy of the group, and y_i is the accuracy of the i-th element of group g.

We also define a weaker null hypothesis: that pairs $(A_1, \sigma_1, A_2, \sigma_2)$ with equal total SNR and equal SNR on each feature yield the same accuracy distribution.

This second hypothesis is useful when the strong one fails (the CNN is not the optimal matched filter in 2D): if we keep fixed the information partition on the two features, the SNR still remains a good measure.

$$\chi^2 = \sum_g \sum_i \frac{(y_i - \mu_g)^2}{\sigma_i^2 + \sigma_g^2} \tag{6}$$

where g stands for the group of points with the same total SNR and the same SNR on each feature, μ_g is the mean accuracy of the group, and y_i is the accuracy of the i-th element of group g.

Performance of the CNN in terms of convergence We create the datasets as explained above. We build and train a CNN model over each dataset until it reaches either a target arbitrary accuracy or the maximum number of iterations, in our case 350. This value is much larger than what is usually used for training a CNN (usually 100), to claim with certainty that after 350 iterations (epochs) the model will not converge to that accuracy. We also make sure that the models reach the target accuracy and stay above the threshold for at least 5 epochs.

Test generalization capacity varying SNR – We create six 2-D datasets using different amplitudes and, thus, different SNR. We explore the SNR space from 0.1 to 30 on a logarithmic scale. Once we train the model, we validate it on a set with the same SNR value. We test each trained model on datasets with different SNR values, in order to see which one can perform better in a more general case. Measurements are performed ten times, and the final result is taken as the mean value obtained, while the variance is used for statistical purposes. The results are presented through heatmap representations.

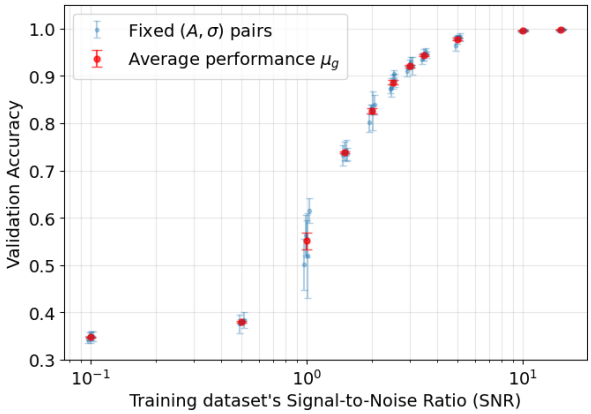


FIG. 1: Model's validation accuracy vs training dataset's SNR. Each blue point's position and error is computed as the average and stdev of the validation accuracies of 5 different models trained on datasets with the same parameters (A, σ).

Effect of regularization on learning Arpit et al. [1] demonstrated that regularization can reduce training performance on datasets containing random labels while preserving generalization on real-world data. In particular, they observed that regularization slows the memorization of noisy data without substantially affecting learning from genuine data.

We generate a dataset with $SNR = 3$ in the usual way (*real data*) and a dataset with the same data points but random labels (*noise data*). We train the same convolutional architecture with different types of regularization (L1, L2, L1L2) varying the regularization strength ($\lambda \in 10^{-6}, ..., 10^{-1}$). For each configuration we train two instances of the model: one on the dataset with the true labels and the other on the one with random labels. We then plot validation accuracy on real labels (*generalization*) against training accuracy on random labels (*memorization*).

RESULTS

Validating the SNR as a measure of learning potential – The test failed to reject the null hypothesis, confirming that the computed SNR is a reliable measure of the dataset's learning potential (Figure 1, Table I). The low χ^2 values suggest that the errors are overestimated, likely due to early stopping at a fixed number of

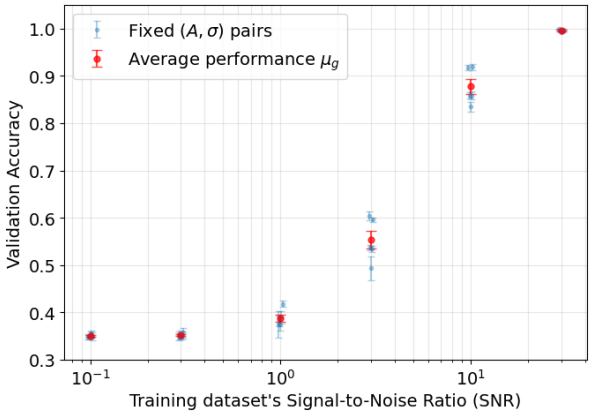


FIG. 2: Model's validation accuracy vs training dataset's SNR. Each blue dot is computed as in Fig. 1 and represents a different partition of the information (measured with SNR) over the two features.

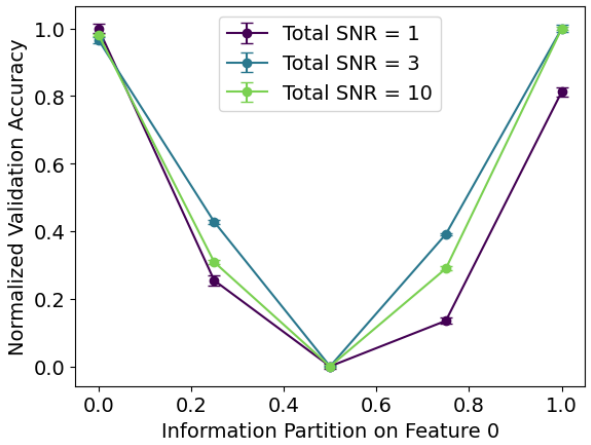


FIG. 3: Normalized accuracy vs. information partition over the two features. The variation depends on the information partition and behaves as a systematic error.

epochs (100) before full convergence.

σ_g	χ^2	$DoF = \mathbb{E}[\chi^2]$
Computed as mean's error	29	44
Set to Zero	38	44

TABLE I: Chi-square test on the hypothesis: Pairs (A, σ) with equal SNR yield the same accuracy distribution

Generalization of the validity of SNR in higher dimension feature spaces – The test rejects the strong null hypothesis with a probability higher than 99.5% (Table II). We see that the points that contribute the most to the hypothesis rejection are the ones where we observe the maximum test accuracy variation (SNR $\in [1, 10]$).

σ_g	χ^2	$DoF = \mathbb{E}[\chi^2]$
Computed as mean's error	60	24
Set to Zero	301	24

TABLE II: Chi-square test on the hypothesis: Pairs $(A_1, \sigma_1, A_2, \sigma_2)$ with equal SNR yield the same accuracy distribution

In Figure 3 we show that the deviation from the mean is systematic and depends only on the information partition between the two features. This, and the failure to reject the weak null hypothesis (Table III), confirm the fact that, for a fixed information partition, the SNR is still a valid measure for both learning potential and test accuracy.

A possible explanation for the null hypothesis rejection is the CNN's *Winner-take-all* behavior: since j_0 is different in the data generation for the two channels and the CNN we are using is not optimized for that, the complexity of the problem is enhanced. To face the task, the CNN gives a weight to the two features based on how much information they carry and, based on that, it performs a classification task primarily on the channel which brings more information.

To quantitatively support the winner-take-all hypothesis, we evaluate the drop in accuracy obtained by zeroing out one channel at a time. The results in Table IV confirm that the CNN progressively shifts its reliance toward the more informative channel rather than optimally combining both features. This explains the systematic deviation from the ideal matched filter behavior observed in the strong hypothesis test.

σ_g	χ^2	$DoF = \mathbb{E}[\chi^2]$
Computed as mean's error	21.94	45
Set to Zero	64.8	45

TABLE III: Chi-square test on the hypothesis: Pairs $(A_1, \sigma_1, A_2, \sigma_2)$ with both equal total and single-feature SNR yield the same accuracy distribution

TABLE IV: Feature 0 contribution to the total SNR (α) vs accuracy drop after removing one feature. The total SNR is fixed at 3.

α	Drop w/o Feature 0	Drop w/o Feature 1
0.00	0.004	0.287
0.25	0.009	0.200
0.50	0.116	0.140
0.75	0.220	0.014
1.00	0.274	0.011

Test generalization capability of the SNR – As shown in the diagonal of the heatmap in Fig. 6a , the ac-

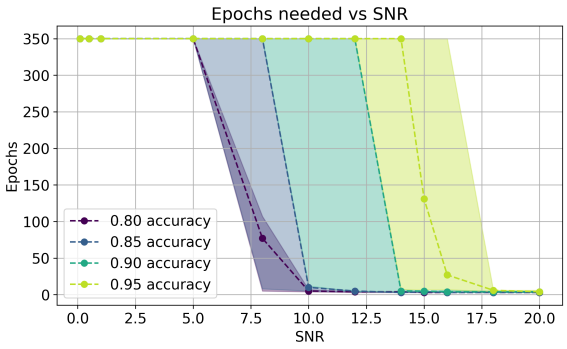


FIG. 4: Epochs needed to reach target accuracy vs SNR.

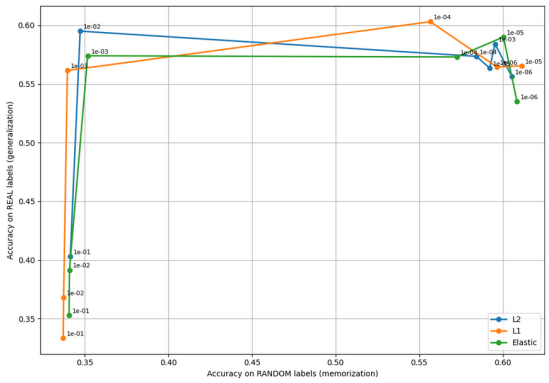


FIG. 5: Effect of different regularizers on train accuracy (on noise dataset with random labels) versus validation accuracy (on real dataset)

curacy computed on a test dataset having the same SNR as that of the training set grows with the SNR. This is not the case for the off-diagonal values; models trained with a dataset having an high SNR value, have rapidly decreasing performances on dataset with smaller SNRs. Intermediate values of SNR appear to be the best compromise of performance and generalization over datasets with different SNRs. We compute the mean of the accuracies of each model tested over all the others:

SNR:	0.1	0.3	1	3	10	30
Mean value:	0.34 ± 0.006	0.35 ± 0.005	0.38 ± 0.008	0.52 ± 0.05	0.50 ± 0.01	0.41 ± 0.006

The best mean value overall is the one corresponding to SNR three; the only measure gaussian compatible with it is the one associated with SNR equal to ten. Thus, it is reasonable to better understand what happens in the section between three and ten. in Fig. 6b, we explore the SNR space between two and ten. The found result is that the true SNR value that better generalizes the performances is 6.

SNR:	2	4	6	8	10
Mean value:	0.52 ± 0.01	0.63 ± 0.007	0.64 ± 0.005	0.63 ± 0.005	0.60 ± 0.005

Performance of the CNN in terms of convergence We notice that for each accuracy target there is a specific SNR cutoff value for which the CNN starts to converge spending less than 50 epochs to reach the accuracy target. We fit the points that have converged with a sigmoid function to get an estimate of the SNR cutoff value.

Accuracy:	0.80	0.85	0.90	0.95
Empirical cutoff:	8	10	14	15
Estimated cutoff:	7.72 ± 0.01	8.47 ± 0.06	11.53 ± 0.37	14.62 ± 0.19

TABLE V: Empirical and estimated SNR cutoff for the target accuracy.

This result is coherent with theoretical claims found in [3]. There exists a lower geometrical threshold based on the Fisher information matrix such that if the model satisfies the PL inequality the model's gradient descent will have linear convergence, and the number of epochs needed for the model to reach the target accuracy will drop dramatically. This inequality depends on the minimum value of the eigenvalues of the Fisher information matrix, so when the noise suppresses the signal in the SNR there is no possibility for the model to learn the pattern from a geometrical point of view. Two ways to lower the cutoff (and consequently the threshold) are to improve the model and to improve the quantity of the data, as the lower limit of the PL inequality is proportional to $\frac{\sqrt{d}}{n}$, where d is the number of parameters of the model and n the number of data fed to the model.

In the case when the inequality is satisfied, the model needs at most 50 epochs to reach over 90% accuracy, confirming the fact that when the model converges, it does so very rapidly.

Effect of regularization on learning Fig. 5 partially reproduces the result of Arpit et al. [1]. As we can see, the effect of regularization on the relationship between memorization and generalization depends on the type of regularizer and its strength. At moderate regularization strengths ($\lambda \leq 10^{-4}$), the models with all three types of regularizers achieve a comparable generalization while exhibiting high memorization. For higher values of the regularization strength ($\lambda > 10^{-4}$) memorization is substantially reduced, which is consistent with what is stated in the article: regularization impairs noise fitting preventing noisy data memorization. However, the effect on generalization is not uniform, as for some types of regularizers and λ the validation accuracy on real data drops as well.
Thus, the memorization-generalization decoupling is regularizer and strength dependent and does not hold uniformly across all penalty types. We still obtain flat curves, meaning that regularization can reduce memo-



(a) (a) (b) (b)

FIG. 6: (a) Cross-SNR accuracy heatmaps showing the performance of models trained on datasets with fixed SNR values and tested across datasets with different SNR conditions. The SNR values range from 0.1 to 30 on a logarithmic scale in (a), and from 2 to 10 in (b). Each experiment is repeated ten times; the reported values correspond to the mean accuracy, while the variance across repetitions is also indicated in the heatmaps.

rization on noisy datasets, while resulting in the same validation accuracy on the corresponding real dataset.

CONCLUSIONS

We have demonstrated that the SNR is a solid measure for the dataset's learning potential and the CNN's validation accuracy. We used this measure to understand how a CNN is capable of generalizing over datasets with lower and higher SNRs. Particularly, we found that there exists an intermediate value of the SNR that provides the best generalization when testing over different datasets. While the accuracy tends to increase with increasing SNR, the ability of the model to generalize does not follow the same trend.

This result suggests that different training strategies should be adopted depending on the application. If the objective is to maximize performance on a specific dataset, training with higher SNR values may be preferable. On the other hand, if robustness and cross-dataset generalization are more important, selecting a dataset with an intermediate SNR may lead to better results. Thus, we provide a possible criterion for choosing the training dataset for the CNN.

There is a lower limit on how small the signal can be with respect to the noise, for the model to learn the pattern and achieve a good overall accuracy. The Fisher Information provides the threshold based on its spectre. This threshold can be lowered by improving the perfomance of the model, by removing parameters on the model or adding more data. This is a trade-off between how simple the model can be and the accuracy of the model, and mark the need of an ideal infinite pool of data, realistically impossible.

The effect of regularization on the memorization–generalization relationship is strongly dependent on both the type of regularizer and its strength. While moderate regularization preserves generalization performance despite high memorization, stronger penalties significantly reduce memorization by limiting noise fitting. However, this reduction doesn't always lead to stable validation accuracy, meaning that the generalization behavior varies across regularization settings.

CODE

The codes could be found at: https://github.com/FrancescoPeron/LCP_B/tree/main/ASSIGNMENT

CONTRIBUTION STATEMENT

Leonardo Fardin dedicates himself to studying the theory of signal detectors, validates the SNR as a measure of learning potential; Lara Fogarolo implements the data generation and studies the effect of regularization in model training; Giacomo Bregolin studies the capability of different SNR values to generalize over different datasets, finding that a middle value of SNR can generalize better than the others; Francesco Peron looks at CNN's performance in terms of convergence time, finding the SNR cutoff value for the phase transition.

APPENDIX A

Here, we first compute the SNR for the raw sample and then derive the SNR as seen by the optimal *matched filter*. We do this to demonstrate the significant gain in analytical simplicity achieved by treating the Convolutional Neural Network as a matched filter.

The signal at a given time step t can be expressed as:

$$s_t = p_t + n_t$$

$$n_t = B(\mu, \sigma)$$

$$dB = \mathcal{N}(\mu, \sigma)$$

$$p_t = A\sin(\frac{\pi}{T}t)\mathbb{1}_{j,j+T}(t)$$

$$j \sim \mathcal{U}(0, L - T - 1)$$

Where p_t is the pattern, n_t the noise, B stands for Brownian motion, $\mathcal{N}$ the normal distribution and $\mathcal{U}$ the uniform one. The *Signal-To-Noise Ratio* for a sample is:

$$SNR_{sample} = \frac{\bar{P}_{signal}}{\bar{P}_{noise}}$$

The theory tells us that a signal has an energy equal to:

$$E = \int_{-\infty}^{+\infty} x^2(t)\ dt$$

Since the pattern in the signal lasts T time steps we can compute the energy of the pattern as:

$$E_{pattern} = \sum_{t=0}^{T}[A\sin(t\frac{\pi}{T})]^2 \approx \int_0^T [A\sin(t\frac{\pi}{T})]^2\ dt =$$

$$A^2\int_0^T \sin^2(t\frac{\pi}{T})\ dt = \left[\frac{A^2}{2}t - \frac{TA^2}{4\pi}\sin(2t\frac{\pi}{T})\right]_0^T =$$

$$= \frac{A^2}{2}T$$

For the noise:

$$E_{noise} = \int_0^T [Var(x) + \bar{x}^2]\ dt = \int_0^T [Var(x) + \bar{x}^2]\ dt =$$

$$= \int_0^T \sigma^2(j+t) + \mu^2(j+t)^2\ dt = \left[\frac{\sigma^2}{2}(j+t)^2 + \frac{\mu^2}{3}(j+t)^3\right]_0^T =$$

$$= \sigma^2\frac{(j+T)^2 - j^2}{2} + \mu^2\frac{(j+T)^3 - j^3}{3} =$$

$$= \sigma^2\frac{T^2 + 2jT}{2} + \mu^2\frac{T^3 + 3jT^2 + 3j^2T}{3}$$

Since j is uniformly distributed:

$$p(j) = \frac{1}{L-T-1}$$

Then:

$$\bar{E}_{noise} = \frac{1}{L-T-1}\int_0^{L-T-1} \sigma^2 \frac{T^2+2jT}{2} + \mu^2 \frac{T^3+3jT^2+3j^2T}{3}\, dj$$

$$M \equiv L-T-1$$

$$\bar{E}_{noise} = \frac{1}{M}\int_0^{M} \sigma^2 \frac{T^2+2jT}{2} + \mu^2 \frac{T^3+3jT^2+3j^2T}{3}\, dj =$$

$$= \frac{1}{M}\left[\sigma^2 \frac{T^2 j + j^2 T}{2} + \mu^2 \frac{jT^3 + \frac{3}{2}j^2T^2 + j^3T}{3}\right]_0^M =$$

$$= \frac{1}{M}\left[\sigma^2 \frac{T^2 M + M^2 T}{2} + \mu^2 \frac{MT^3 + \frac{3}{2}M^2T^2 + M^3T}{3}\right] =$$

$$= \sigma^2 \frac{T^2+MT}{2} + \mu^2 \frac{T^3 + \frac{3}{2}MT^2 + M^2T}{3} =$$

$$= T\left[\sigma^2 \frac{T+M}{2} + \mu^2 \frac{T^2 + \frac{3}{2}MT + M^2}{3}\right]$$

$$= T\left[\sigma^2 \frac{L-1}{2} + \mu^2 \frac{(L-1)^2 - \frac{1}{2}MT}{3}\right]$$

The sample's SNR is then:

$$SNR_{sample} = \frac{A^2 T}{2T\left[\sigma^2 \frac{(L-1)}{2} + \mu^2 \frac{(L-1)^2 - \frac{1}{2}MT}{3}\right]} =$$

$$\frac{A^2}{\sigma^2(L-1) + \mu^2\left(\frac{2(L-1)^2}{3} - \frac{MT}{3}\right)}$$

$$a = (L-1)$$

$$b = \frac{2(L-1)^2}{3} - \frac{MT}{3}$$

$$\boxed{SNR_{sample} = \frac{A^2}{a\sigma^2 + b\mu^2}} \tag{7}$$

The output of a Matched Filter at time T is given by:

$$Y = \int_0^T s(t)p(t)\, dt = \int_0^T p^2(t)\, dt + \int_0^T n(t)p(t)\, dt$$

$$S^2 = \int_0^T p^2(t)\,dt = \left(\int_0^T A^2 \sin^2\left(\frac{\pi}{T}t\right) dt\right)^2 = \left(\frac{A^2 T}{2}\right)^2 = \frac{A^4 T^2}{4}$$

The variance of the noise component $N = \int_0^T n(t)p(t)\,dt$ is calculated using the autocovariance of Brownian motion $K(t,u) = \sigma^2 \min(t,u)$:

$$Var(N) = \sigma^2 \int_0^T \int_0^T p(t)p(u)\min(t,u)\,dt\,du$$

$$Var(N) = A^2\sigma^2 \int_0^T \sin\left(\frac{\pi}{T}t\right)\left[\int_0^t u\sin\left(\frac{\pi}{T}u\right)du + t\int_t^T \sin\left(\frac{\pi}{T}u\right)du\right]dt$$

The inner integrals:

$$\int_0^t u\sin\left(\frac{\pi}{T}u\right)du = \frac{T^2}{\pi^2}\sin\left(\frac{\pi}{T}t\right) - \frac{Tt}{\pi}\cos\left(\frac{\pi}{T}t\right)$$

$$t\int_t^T \sin\left(\frac{\pi}{T}u\right)du = \frac{Tt}{\pi}\left(1 + \cos\left(\frac{\pi}{T}t\right)\right)$$

Then:

$$Var(N) = A^2\sigma^2 T^3\left(\frac{\pi+4}{2\pi^3}\right)$$

The Matched Filter's SNR is the ratio:

$$SNR_{MF} = \frac{S^2}{Var(N)} = \frac{\frac{A^4T^2}{4}}{A^2\sigma^2T^3\frac{\pi+4}{2\pi^3}}$$

$$SNR_{MF} = \frac{A^2\pi^3}{2\sigma^2 T(\pi+4)}$$

$$\boxed{SNR_{MF} = \frac{\pi^3}{2(\pi+4)}\frac{\delta}{\sigma^2}} \tag{8}$$

Where $\delta = \frac{A^2}{T}$ contains all the information about the signal.

APPENDIX B

Listing 1: Function used to build the CNN used for our analysis

```
def build_model(L, l, r, n_class=3, kernel_size=11, NF=5, stddev=0.05):

    input_shape = (L, 2)

    if r == 2:
        reg = tf.keras.regularizers.l2(l)
    elif r == 1:
        reg = tf.keras.regularizers.l1(l)
    elif r == 0:
        l2 = np.abs(1-l)
        reg = L1L2(l, l2)

```

```
13
14      ini = tf.keras.initializers.RandomNormal(mean=0.0, stddev=stddev, seed=None)
15
16
17      model = Sequential([
18          Conv1D(filters=NF, kernel_size=kernel_size,
19                 kernel_initializer=ini,
20                 kernel_regularizer=reg,
21                 padding='same',
22                 activation='relu',
23                 input_shape=input_shape),
24          MaxPooling1D(pool_size=2), #we need to catch a signal that is above the noise
25          Conv1D(filters=NF * 2, kernel_size=5, activation='relu'),
26          MaxPooling1D(pool_size=2),
27          Flatten(),
28          Dense(32, activation='relu'),
29          Dropout(0.2),
30          Dense(n_class, activation='softmax')
31      ])
32
33      model.compile(
34          loss=keras.losses.categorical_crossentropy,
35          optimizer=tf.keras.optimizers.Adam(),
36          metrics=['accuracy']
37      )
38
39      return model
```

[1] D. Arpit, S. Jastrzębski, N. Ballas, D. Krueger, E. Bengio, M. S. Kanwal, T. Maharaj, A. Fischer, A. Courville, Y. Bengio, and S. Lacoste-Julien, *A Closer Look at Memorization in Deep Networks*, arXiv:1706.05394v2 (2017).

[2] J. Yan, M. Avagyan, R. E. Colgan, D. Veske, I. Bartos, J. Wright, Z. Márka, and S. Márka, *Generalized Approach to Matched Filtering using Neural Networks*, arXiv:2104.03961v3 (2022).

[3] William Hao-Cheng Huang *Spectral Thresholds for Identifiability and Stability: Finite-Sample Phase Transitions in High-Dimensional Learning*, arXiv:2510.03809v2 (2025).